॥श्रीगणेशाय नमः॥

# Adversarial Search

## Game theory

- A branch of mathematics that is concerned with the analysis of strategies for dealing with competitive situations where actions taken by one will affect others' outcomes.
- The focus of playing interactively will also ensure that a player has a rational mindset (how she can maximize the expected value of the objective fn).
- It is assumed that the players within the game are rational and will strive to maximize their payoffs.

Game representation:
- who the players are.
- rules of the game/ actions available to the game.
- knowledge given to the players about the outcomes.
- strategies.

A game is a formal representation of this information.

a. Extensive form representation
b. Normal form representation

- Game tree
- Information set
- allocation of non-terminal nodes of tree to player
- payoffs for each player at terminal nodes

Nash equilibrium:
- Game reached on an outcome.
- When each player says, "Given the moves of the opponent, I'm happy with my choices".
- It doesn't mean best outcomes.

Prisoner's dilemma:
- Why two completely rational individuals might not cooperate, even if it appears that it is in their best interests to do so.

# Minimax search

2-player zero-sum games:
- These are deterministic.
- These involve only 2 players.
- These are played in a perfect information environment (can observe opponents' moves, nothing hidden).
- Zero-sum game: no win-win game.

## Formal definition of a game

- Initial state.
- Next function (state) → list of neighbors [(move, state)]
- Is_terminal (state) → True (leaf) or False (non-leaf)
- Utility (state): An objective function or a payoff function. It defines the final numeric value for a player p when the game ends in a terminal state s.

We'll be defining the complete game tree as a search tree that follows every sequence of moves all the way to the terminal node.

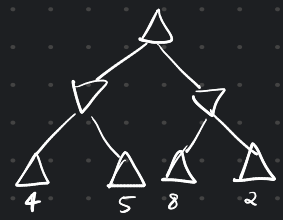
## Minimax strategy

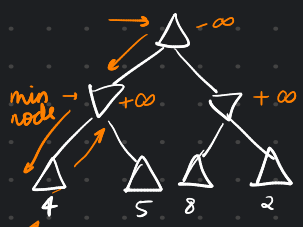
- A good first move will be fetched by the minimax function.
- Generating the complete game tree starting from max's perspective.
- Compute the payoffs or utilities associated with every terminal node.
- Back them up level-by-level.
- Decide which path to take.

△ □ } max node → tries to maximize

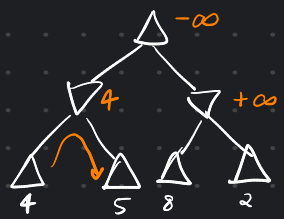
▽ ○ } min node → tries to minimize

Solved

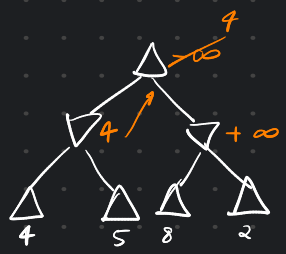


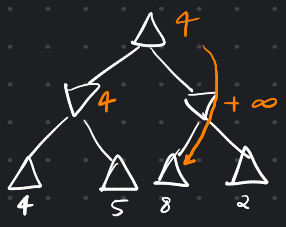

leaf node is smaller than parent.
back up.



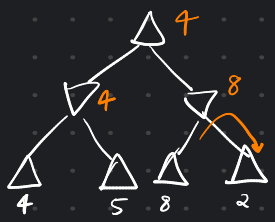

5 is not less than the current value.
Do not back up.



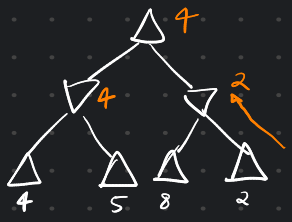

4 > -∞
replace.
we want to maximize the max nodes.



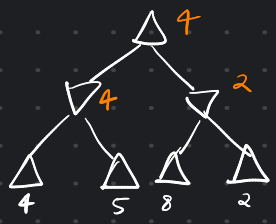

8 < +∞
replace

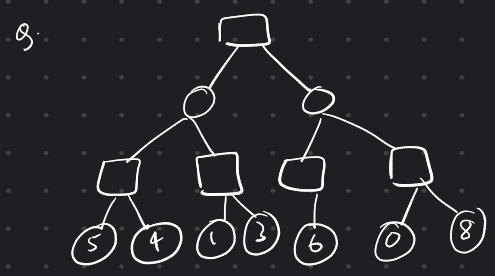


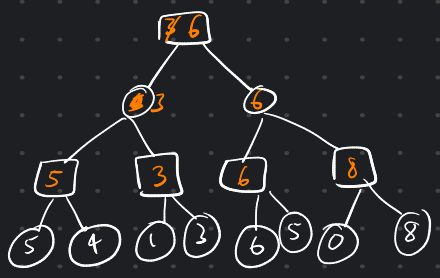

2 < 8
back up




2 > 4 ×
do not replace.







## Performance metrics

Complete? Yes. Given a finite game tree

Time? $O(b^m)$

Space? $O(bm)$

Optimal? Yes, if P2 plays optimally.

Drawbacks
- Not possible to expand a game with all possibilities.

---

```python
def MINI_MAX(state):
    v = MAX_VALUE(state)
    return v.move
```

```python
def MAX_VALUE(state):
    if IS_TERMINAL(state):
        return UTILITY(state)
    v = '-infty'
    for v` in neighbors(v):
        v2 = MIN_VALUE(v`)
        if v2.value > v.value:
            v = v2
    return v
```

```python
def MIN_VALUE(state):
    if IS_TERMINAL(state):
        return UTILITY(state)
    v = '+infty'
    for v` in neighbors(v):
        v2 = MAX_VALUE(v`)
        if v2.value < v.value:
            v = v2
    return v
```

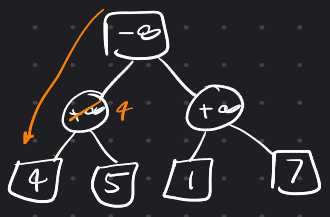
```
function MINIMAX-SEARCH(game, state) returns an action
  player ← game.TO-MOVE(state)
  value, move ← MAX-VALUE(game, state)
  return move

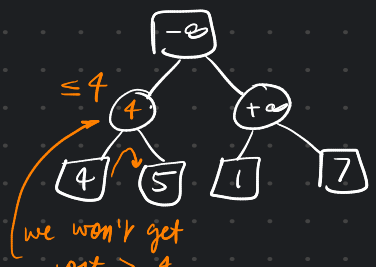
function MAX-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← −∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a))
    if v2 > v then
      v, move ← v2, a
  return v, move

function MIN-VALUE(game, state) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a))
    if v2 < v then
      v, move ← v2, a
  return v, move
```

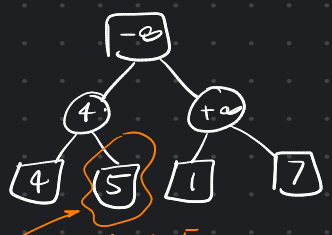
# Alpha-beta pruning

minimax(root) = max(min(4,5), min(1,x))



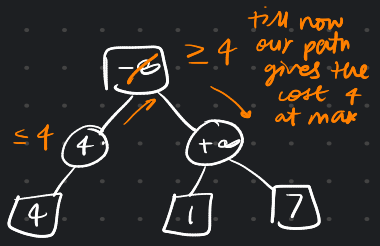




we won't get cost > 4 at this point even if we won't check further.

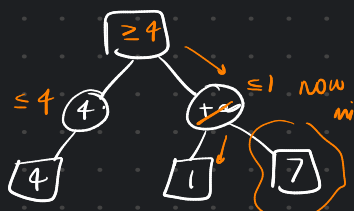



Now we see 5. we don't back it up

Prune this sub-tree. In certain situations, this will reduce the search a lot.



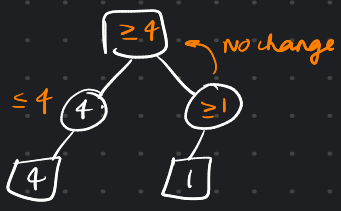

till now our path gives the cost 4 at max



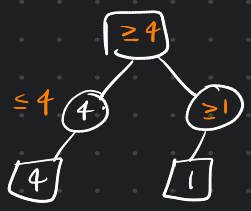

now since this is a min-node, we'll get 1 or something less, no matter what the right child is.

And so the parent max node will still consider 4.

this region is not even checked. sub-tree is pruned.




no change

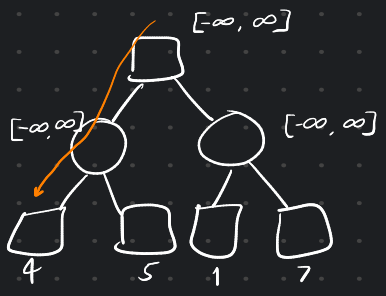


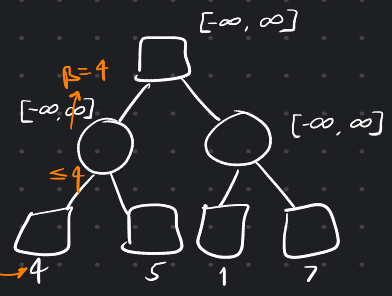

αβ pruning is an improvement to minimax.
It reduces the search time by pruning parts of the search tree.

Every node has α and β values.

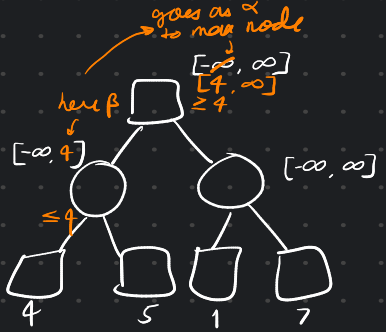
α → Best value for **max** along the path  $v \geq \alpha$
β → Best value for **min** along the path  $v \leq \beta$

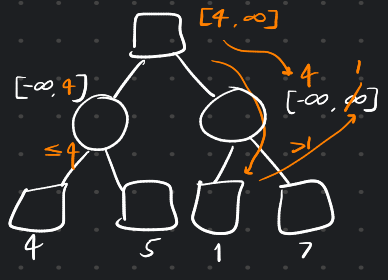
} only then do we care

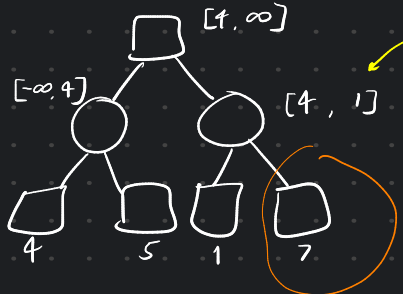



4 is min, upper node is max. so max value we get from this path is 4.



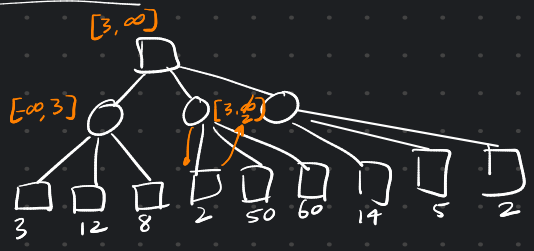



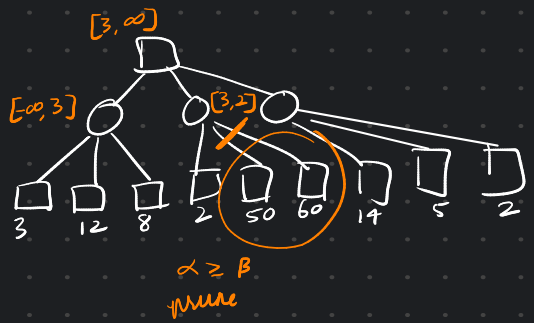



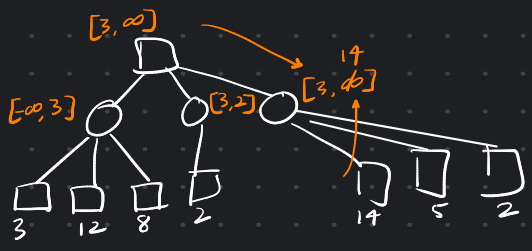

condition for pruning : $\alpha \geq \beta$

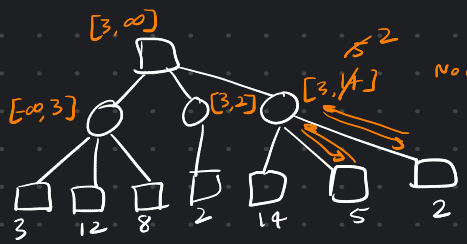
prune since $\alpha \geq \beta$

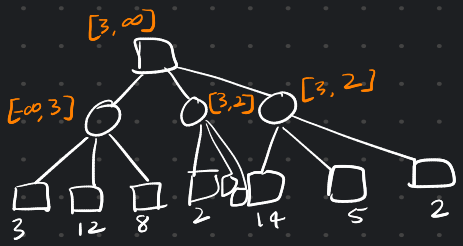
another example:



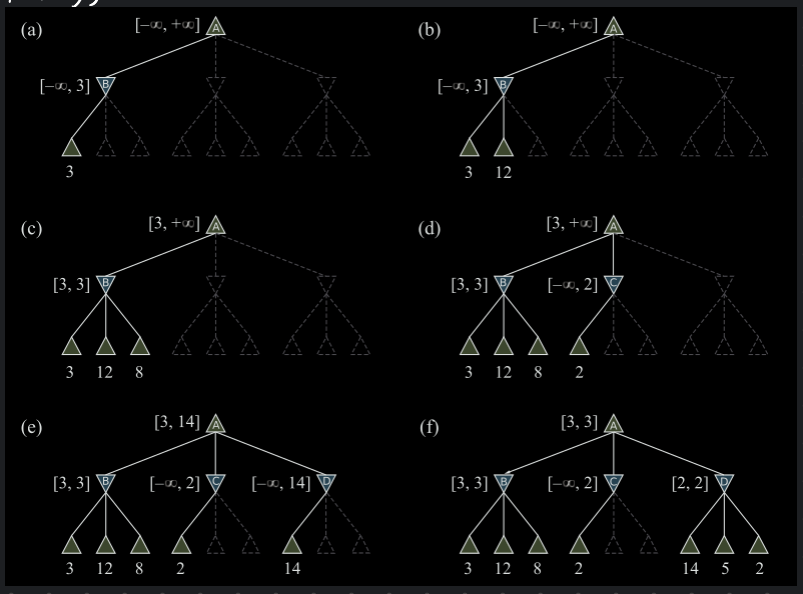



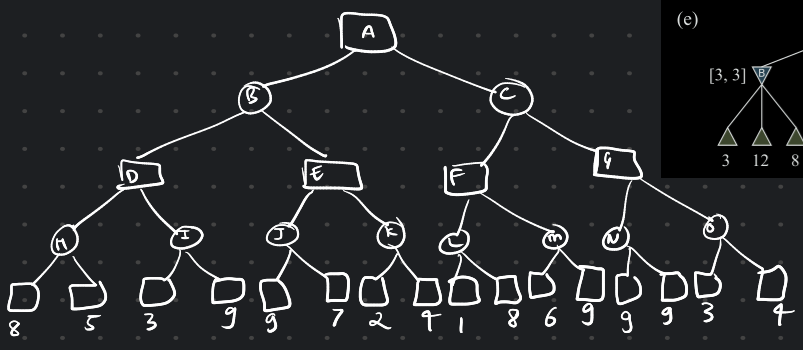



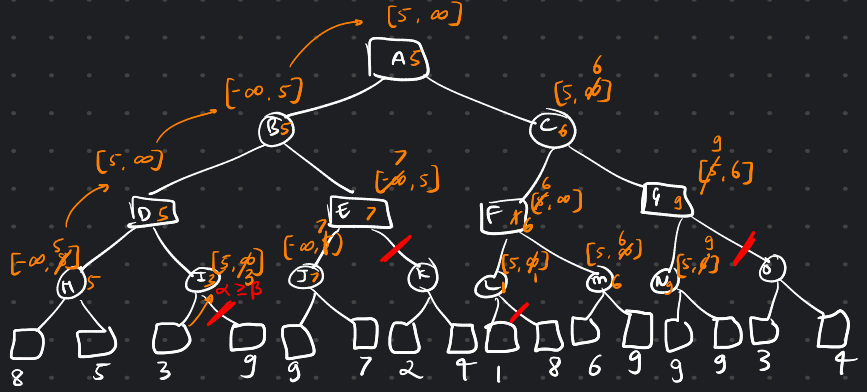



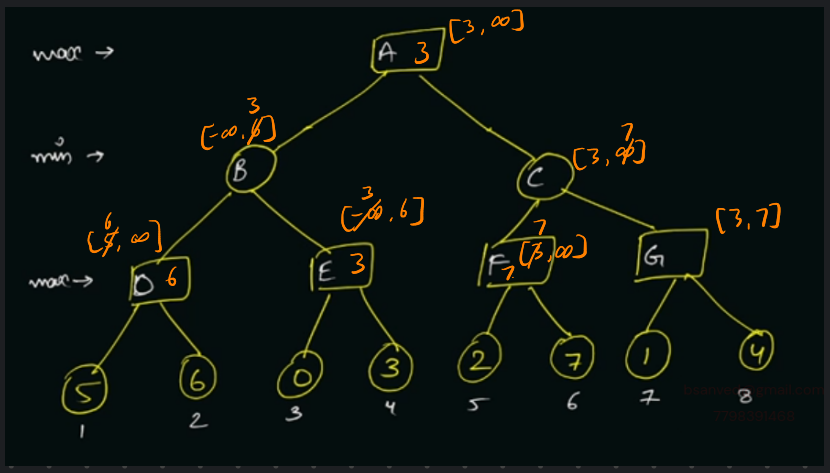

minimax(root) = max(min(3,12,8), min(2, x, y), min(14,5,2))
= max(3, ≤2, 2)
= 3




9.



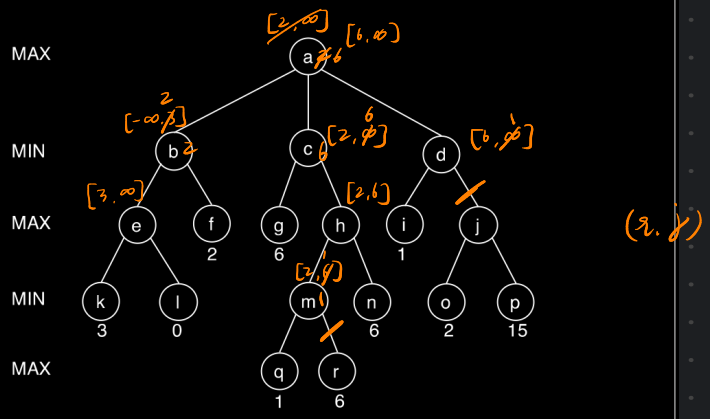



Here α β pruning won't prune anything.

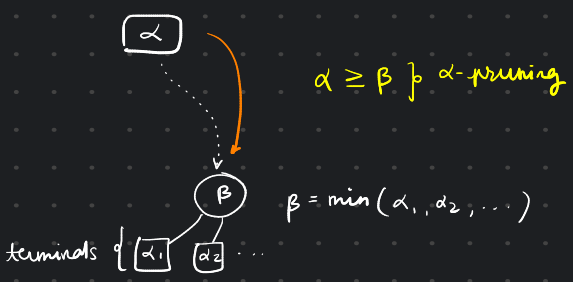
Worst case → $O(b^m)$

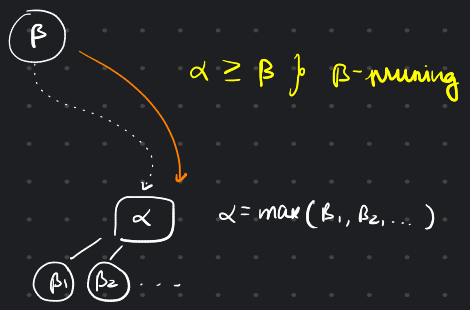
Best case → $O(b^{m/2})$

| | |
|---|---|
| Q.31 | Consider the game tree shown below. The value below each node is the output of the utility function. The subtrees rooted at which of these nodes will be pruned because of alpha-beta pruning? |



# Alpha and beta cutoffs




```
function ALPHA-BETA-SEARCH(game, state) returns an action
  player ← game.TO-MOVE(state)
  value, move ← MAX-VALUE(game, state, −∞, +∞)
  return move

function MAX-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← −∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a), α, β)
    if v2 > v then
      v, move ← v2, a
      α ← MAX(α, v)
    if v ≥ β then return v, move
  return v, move

function MIN-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a), α, β)
    if v2 < v then
      v, move ← v2, a
      β ← MIN(β, v)
    if v ≤ α then return v, move
  return v, move
```

## Heuristic αβ search

$$\text{minimax}(s) = \begin{cases} \text{utility}(s, \text{max}) & \text{if IS-TERMINAL}(s) \to T \\ \max(\text{minimax}(s \to a)) & \text{to go towards max} \\ \min(\text{minimax}(s \to a)) & \text{to go to min} \end{cases}$$

we will cut off the search early and apply a heuristic evaluation function treating non-terminal nodes as if they are terminal.

we'll replace the utility → eval → which estimates a state's utility.

terminal test → cutoff test.

$$\text{heuristic minimax}(s) = \begin{cases} \text{eval}(s, \text{max}) & \text{if IS-CUTOFF}(s, \text{depth}) \\ \max(\text{minimax}(s \to a)) & \text{to go to max} \\ \min(\text{minimax}(s \to a)) & \text{to go to min} \end{cases}$$

eval $(s, p)$ → terminal → utility$(s)$

→ non-terminal state → between a loss and a win.

$$\text{utility}(\text{loss}, p) < \underline{\underline{\text{eval}(s, p)}} < \text{utility}(\text{win}, p)$$

↳ finding this must not take long.
- eval should also correlate with chances of winning.

It might be possible to prune a non-terminal state that will leading to a winning terminal node.
↳ optimality is compromised.

## Forward pruning

Prune the moves that appear to be poor.

# Monte carlo tree search

It is a specific variant of Monte Carlo search often used in adversarial games.




4-step process:

① Selection: Given we are at a node, selecting which path to take given n choices. The selec" is based on some selection policy.

② Expansion: Growing the game tree by generating a new child of the selected node. (0/0)

③ Simulation: From the new node, choosing moves for both players accd to a playing policy. It can end on a terminal node or upto next n moves ∈ (50, 100, smth)

④ Back propagation: Using the result of simulation to update all the search tree nodes going up to the root.

(x/y) → That node has won x out of y gameplays.




Selection policy used in MCTS:

- Upper confidence bounds, $UCBI(n) = \frac{U(n)}{N(n)} + C * \sqrt{\frac{\ln(parent(n))}{N(n)}}$

exploitation

exploration

Lesser the value of $\sqrt{N}$, more the value of exploration part.

$\frac{x}{y} + C\sqrt{\frac{\ln(parent\ x)}{y}}$

$U(n)$ → x for node n

$N(n)$ → y for node n

$Parent(n)$ → x for n's parent

$C$ → constant for weight to exploration part.